

Pi-hole — network-wide DNS ad-blocking

What Pi-hole is, how it's wired into the k3s home-cluster, what it actually does to ads and trackers, and — just as important — the ways a DNS sinkhole can quietly break things. A plain-language brief for a single-operator home network.

Date · 2026-06-30**Version** · Pi-hole FTL v6.6.2**Runs on** · k3s · node a2**Admin** · <https://pihole.lan/admin>

At a glance

53**DNS PORT**

192.168.5.96:53
tcp+udp on a2

82,755**DOMAINS BLOCKED**

StevenBlack default list

0.0.0.0**SINKHOLE REPLY**

Ad domains answered
with a dead address

1**REPLICAS**

Single pod — no DNS
failover yet

In one sentence — it's a DNS server that lies about ad domains

Pi-hole sits between your devices and the internet's DNS. For ordinary names it forwards the question upstream and returns the real answer. For names on its blocklist — ad servers, trackers, telemetry endpoints — it returns a **dead address** (0.0.0.0), so the browser asks for the ad, can't find the server, and silently gives up. Nothing is "stripped" from pages; the requests simply never connect.

What it is (and isn't)

What it is

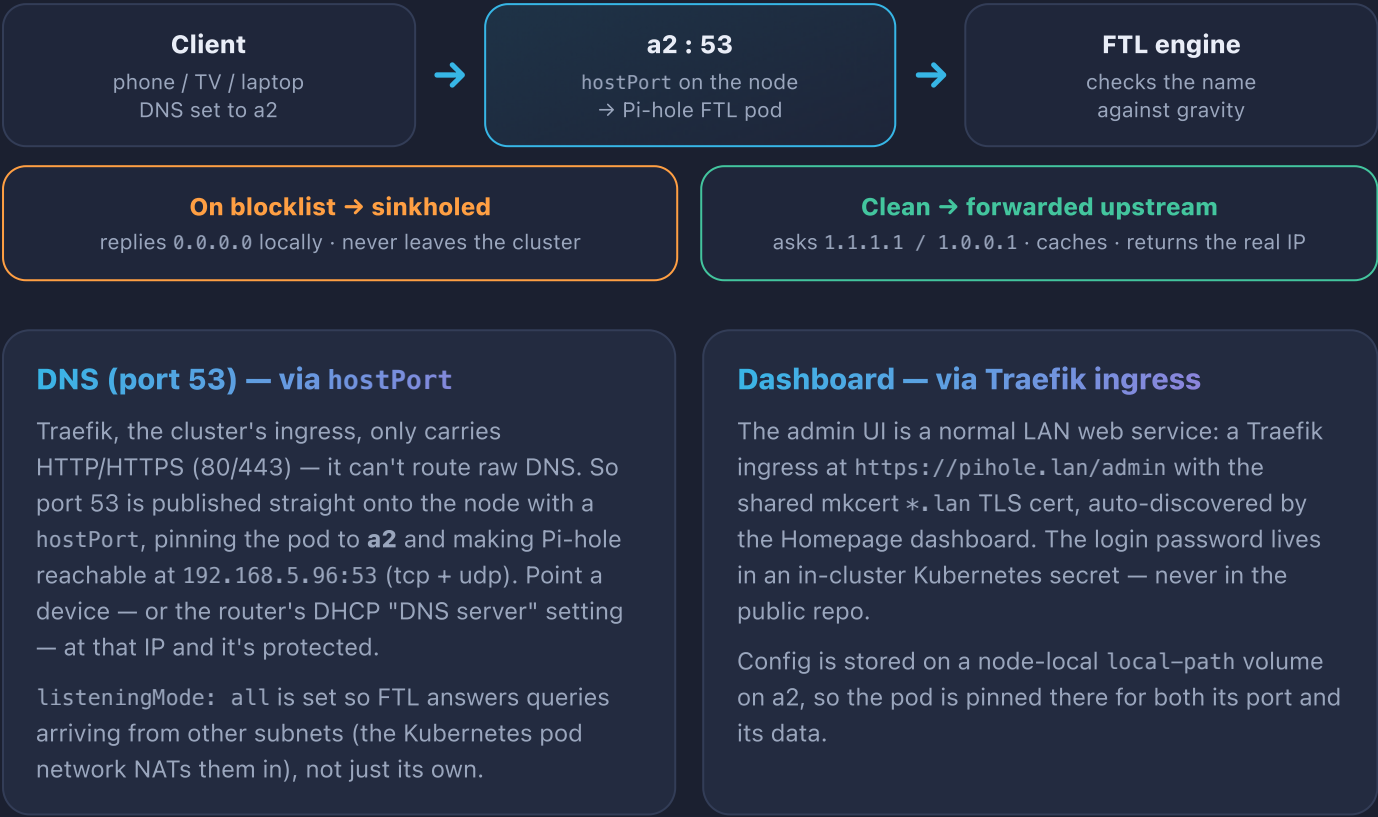
- ▶ A recursive-ish **DNS forwarder** with a blocklist ("gravity")
- ▶ Network-wide: protects **every device** that uses it for DNS — phones, TVs, IoT — with no per-device software
- ▶ A **local resolver + dashboard** showing every query your network makes
- ▶ Open-source; here it runs as one container (FTL = the DNS engine + web UI)

What it is not

- ✗ Not a firewall, VPN, or content filter for HTTPS payloads
- ✗ Not an in-browser ad-blocker — it can't hide page elements, only block network requests
- ✗ Not able to block ads served from the **same domain** as the content (e.g. first-party / YouTube)
- ✗ Not effective against apps that use their own hardcoded DNS or DNS-over-HTTPS

The networking around it

Pi-hole has two front doors with very different needs: a **DNS service on port 53** that the whole LAN must reach, and a **web dashboard** on HTTP. They're exposed two different ways.



Why a2, not the control-plane (a3)

FACTOR	A3 (CONTROL-PLANE)	A2 (CHOSEN)
Uptime profile	Always-on — ideal for DNS	Up reliably; the primary workload node
Network link	Wired	Wired
Free disk	~75 GB	~527 GB
Pulls images from docker.io	Currently unreliable (resolver issue)	Healthy

a3 was the natural "always-up" home for DNS, but its container runtime currently can't reliably resolve docker.io to pull images — a separate node-level issue tracked as debt. a2 pulls cleanly, is wired, has the most headroom, and already hosts most LAN services, so Pi-hole lives there for now.

What it does about ads & trackers

DNS sinkholing, step by step

- ▶ A page loads and tries to fetch a banner from, say, `doubleclick.net`.
- ▶ The browser first asks DNS, "what's the IP for `doubleclick.net`?" — and that question goes to Pi-hole.
- ▶ Pi-hole finds the name on its **gravity** blocklist and answers `0.0.0.0` instead of the real IP.
- ▶ The browser tries to connect to `0.0.0.0`, which goes nowhere, so the ad request **fails instantly** and the slot stays empty.
- ▶ Multiply that across **82,755** known ad/tracker domains and most display ads, trackers, and telemetry never load.

What this is good at

- ▶ **Trackers & telemetry** — analytics and "phone-home" beacons on dedicated domains
- ▶ **Display / banner ad networks** — served from third-party ad domains
- ▶ **Smart-TV & IoT spying** — devices you can't install a blocker on
- ▶ **Speed & data** — blocked requests are never downloaded at all
- ▶ **Whole-network reach** — one config covers every device

What it can't catch

- ✗ **First-party ads** — when ads come from the same domain as the content (YouTube, many apps), blocking the domain breaks the whole service, so it isn't blocked
- ✗ **In-page elements** — it works at the domain level, so it can't selectively hide a div or skip a pre-roll
- ✗ **Hardcoded resolvers / DoH** — apps that bake in `8.8.8.8` or use DNS-over-HTTPS skip Pi-hole entirely
- ✗ **"Empty box" leftovers** — the ad fails to load but the page may leave a blank rectangle

Because of those gaps, Pi-hole is best thought of as a **network-wide tracker/telemetry blocker** that also removes a large share of display ads — complementary to, not a replacement for, an in-browser content blocker (e.g. uBlock Origin) which *can* reach inside the page.

Blocklists ("gravity")

The block set is just a big list of domains compiled from subscribed **adlists** into a local database called *gravity*. This deployment uses the well-known StevenBlack consolidated hosts list (**82,755** domains). You tune Pi-hole by **adding adlists** (more coverage, more risk of false positives), **allowlisting** domains it shouldn't block, or **denylisting** extra ones by hand. Updating gravity re-pulls the lists.

Possible issues it could create

A DNS sinkhole becomes a piece of **critical infrastructure** the moment devices depend on it — when it misbehaves, "the whole internet is broken" is the usual symptom. The main risks, ranked:

HIGH P1 Single point of failure for all DNS

One pod, on one node (a2), with no second replica. If a2 reboots, the pod crashes, or the volume hiccups, **every device pointed at it loses DNS** — and DNS down feels like the entire network is down. The blast radius is the whole house.

Mitigate: hand out a **second DNS server** in DHCP (e.g. the router or 1.1.1.1) as a fallback; longer term, run a second Pi-hole / HA pair. Don't make it the only resolver for anything you can't afford to lose.

MED P2 Over-blocking & false positives

Aggressive or stacked blocklists inevitably catch domains you actually need — a login that calls an analytics SDK, a payment iframe, a CDN, app-store assets, push notifications. The failure is **silent and confusing**: the site half-loads or a button does nothing, with no error pointing at DNS.

Mitigate: when something breaks, check the **Query Log** for a blocked domain and **allowlist** it. Add adlists conservatively.

MED P3 Upstream & cache dependency

For non-blocked names Pi-hole forwards to an upstream (1.1.1.1). If that upstream or the cluster's egress is slow/unreachable, lookups stall — exactly the cold-start timeout seen right after deploy.

Mitigate: configure **two upstreams**; consider a local recursive resolver (unbound) to reduce external dependence; watch query latency.

LOW P4 Bypass & a false sense of security

Modern browsers and apps increasingly use **DNS-over-HTTPS** or hardcoded resolvers that route *around* Pi-hole, so blocking looks "leaky." And blocking a tracker's DNS name is **not** security — it doesn't stop malware, phishing over allowed domains, or anything already inside an HTTPS session.

Mitigate: disable DoH in browsers (or force them at Pi-hole); treat Pi-hole as privacy hygiene, not a security control.

LOW P5 Operational drift

Stale blocklists, an unbounded query database growing on the local volume, or a manual allowlist that never made it back into the repo. Config lives on a node-local volume, so a lost node means re-creating tweaks.

Mitigate: keep meaningful config as IaC in the repo; schedule gravity updates; back up /etc/pihole; revisit the a3 control-plane image-pull issue so HA is an option.

Bottom line

Pi-hole is a high-value, low-cost privacy win for the whole network — but it turns DNS into something you now **operate**. The two rules that keep it boring: always give clients a **fallback resolver**, and when "the internet is broken," **check the query log first**.